Assignment Report

SFWRENG 2AA4 (2026W)

Assignment 3

Author(s)

Manasvi Bandi	bandim@mcmaster.ca	bandim
Nithya Majeti	majetin@mcmaster.ca	majetin
Mohit Gedela	gedelm1@mcmaster.ca	gedelm1
Lohitashwa Madhan	madhanl@mcmaster.ca	madhanl

GitHub URL

`https://github.com/MohitGedela/Team19-Catan.git`

`https://github.com/MohitGedela/Team19-Catan-V2.git`

`https://github.com/MohitGedela/Team19-Catan-V3.git`

March 20, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Executive summary

This assignment asked us to extend our Catan simulator with machine intelligence and an undo/redo feature. Overall it went reasonably well, though we ran into some challenges along the way, mostly around figuring out which design patterns fit the requirements and how to integrate everything without breaking the existing code.

For R3.1 we implemented undo/redo using the Command pattern, for R3.2 we used the Strategy pattern to give the computer player a rule-based decision system, and for R3.3 we used the Chain of Responsibility pattern to handle game constraints in priority order before the agent makes its move. The three patterns ended up complementing each other nicely since R3.3 runs first and hands off to R3.2 when no constraint applies.

The biggest challenge was making sure R3.3 and R3.2 worked together correctly since they were developed in parallel. Once we sorted out the integration in ComputerPlayer.takeAction(), everything came together. We are happy with how the final design turned out as it is clean, easy to follow, and would not be hard to extend in the future.

2 Requirements traceability

Req ID	Status	Implemented in	Design considerations
R3.1	Implemented	`Command.java`, `CommandHistory.java`, `HumanPlayer.java`	Command pattern for undo/redo. Each action implements execute() and undo(); history is stored on two stacks.
R3.2	Implemented	`Agent.java`, `Rule.java`, `EarnVictoryPoints.java`, `BuildSomething.java`, `SpendingCards.java`	Strategy pattern. Agent evaluates each rule and picks the highest value (1.0, 0.8, 0.5); ties broken at random.
R3.3	Implemented	`ConstraintHandler.java`, `CardLimitHandler.java`, `LongestRoadHandler.java`, `ComputerPlayer.java`	Chain of Responsibility. Handlers linked with setNext(); chain runs before the agent so constraints are resolved before value-based rules.

3 Final UML Diagram

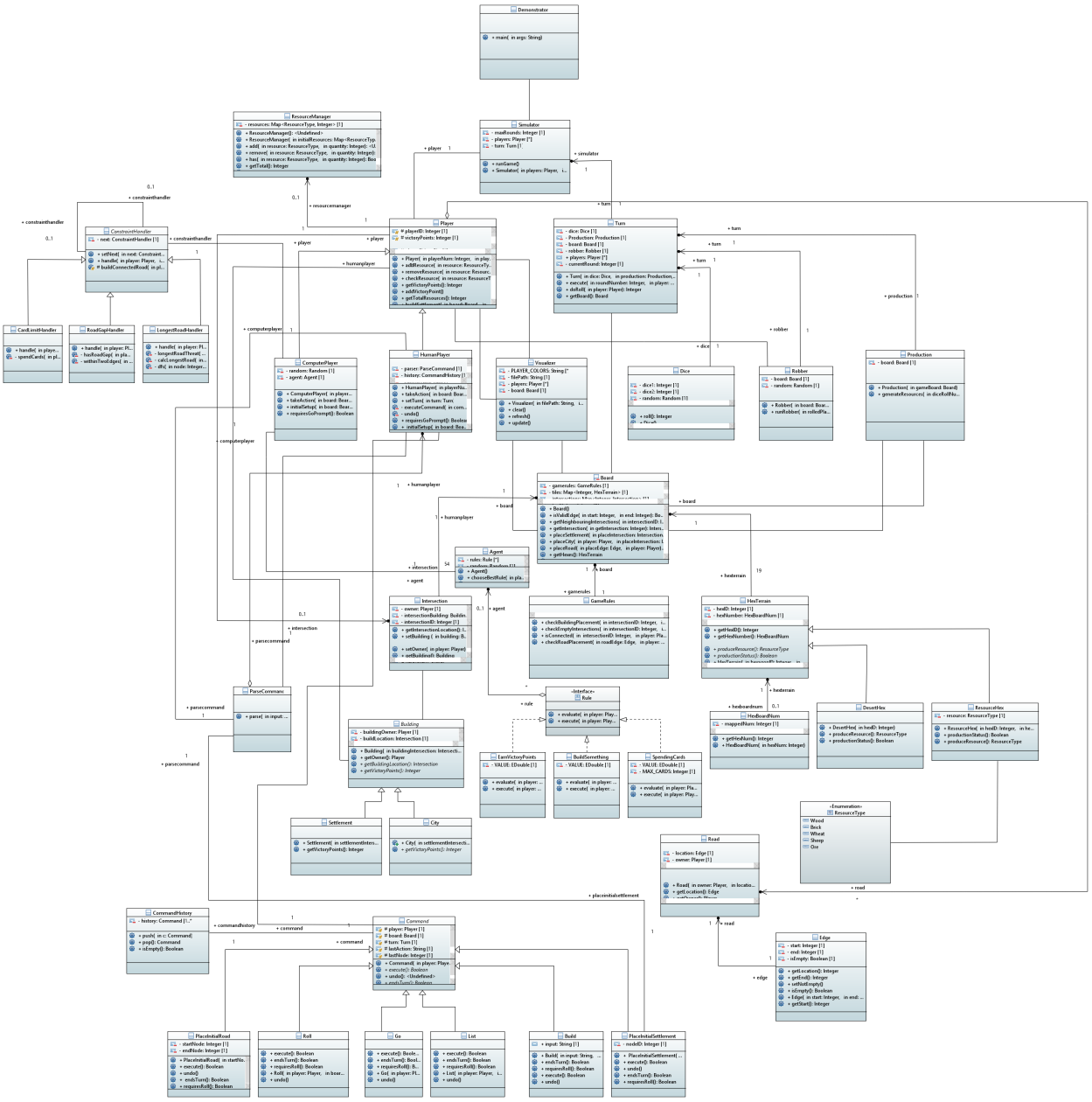

Figure 1: Finalized UML Diagram with all requirements

4 Design pattern 1

Which design pattern did you choose and why?

For Task 1 (R3.1), we decided to go with the Command pattern. The Command pattern is appropriate for the undo/redo feature because any action taken by the user can be represented as an object that knows how to perform the action, as well as undo the action if desired.

We store a history of these command objects, so that pressing Undo will pop the last one off the stack and call undo(), and pressing Redo will just call execute() again. That keeps HumanPlayer as a simple invoker instead of one huge method full of special cases. Adding a new action just requires a new command object and a line of code in the parser, which is easier than modifying if statements. The assignment eliminates Singleton and Factory Method patterns, so Command is a good choice anyway, being a popular pattern for undoable operations.

Explain the design pattern and elaborate on how it fosters cleaner design in this particular software.

The Command pattern converts requests, which the human types, into objects that have a common interface, execute() in this case, along with undo() in a few instances, along with canUndo(), endsTurn(), and requiresRoll() flags. There is no need to replicate the logic of "build settlement" or "place initial road" in the invoker, the HumanPlayer. It is sufficient to parse the input into a command, execute() it, and in a few instances, add it to the CommandHistory. Undoing is just as simple, pop off one stack, execute() or undo() again, without duplicating game logic in the UI loop. Each command, Build, PlaceInitialSettlement, PlaceInitialRoad, etc., is responsible for itself, which is good design, and prevents the game logic in the Board and Player classes from knowing about the command line interface history. ParseCommand encapsulates this, so the big loop remains uncluttered. It is also easier to extend this design, a new action is a new class, a new branch in ParseCommand, not a change to a huge switch statement on strings in the player.

How much did you need to think about the structure of the evolved program once you have figured out which design pattern to use?

Once we settled on Command, a lot of the overall shape was already figured out: we needed an abstract command, concrete commands for each action, an invoker (HumanPlayer), and a history object with two stacks to handle undo/redo. That was the "where things lived" part, but we also needed to think about details such as which commands were undoable vs. read-only (Roll, Go, List), what Build stored enough state to reverse (lastAction, nodes, edges) to allow undoing of settlements, cities, and roads, and how initial placement used the same pattern but didn't violate the setup process. We also needed to think about turn-related state (hasRolled, requiresRoll, endsTurn) to get this loop correct. So, the pattern was helpful in reducing open-ended design work, but we still needed to think about details to get everything just right. It wasn't plug-and-play, so to speak.

How much change was needed in the design and in the implementation?

Design: We went from a state where the human turn was "a bunch of string checks and direct calls into Player/Board" to a more structured command + history model. This translates to design elements such as the responsibilities and rules for the newly created elements like the command abstraction, invoker, undo/redo caretaker, and text to command mapping, and the rules and responsibilities for what is recorded in the history, such as canUndo(). The domain, such as the board, resources, and VP, did not conceptually change, but the location of the undo/redo functionality did, moving into the command abstraction.

Implementation: This was a moderate-sized change, and it was localized to the human path, where we created a Command, CommandHistory, ParseCommand, and various concrete commands such as Roll, Go, ListCommand, Build, PlaceInitialSettlement, PlaceInitialRoad, and refactored the HumanPlayer to use the executeCommand, Undo, and Redo functionality. We extended functionality where needed, such as the Build.undo functionality, which impacts the VP, buildings, and resources, and the Board.removeRoad functionality, which is used in the undo functionality. The ComputerPlayer and the simulation loop were not changed much for R3.1, so the impact is small.

5 Design pattern 2

Which design pattern did you choose and why?

We chose the Chain of Responsibility pattern for implementing the constraint checking logic in R3.3. This pattern was a really good fit because R3.3 requires checking multiple constraints in a very strict priority order and if one constraint applies, it must be resolved before moving on. Each constraint is independent and only matters if the previous one did not apply, which is exactly how a chain works. It also helped that adding or removing a constraint in the future would just mean changing the chain, not rewriting any existing logic.

Explain the design pattern and elaborate on how it fosters cleaner design in this particular software.

Chain of Responsibility works by passing a request through a sequence of handler objects. Each handler decides whether to handle the request or pass it along to the next one. In our case, ConstraintHandler is the abstract base class that manages the chain, and each concrete handler like CardLimitHandler and LongestRoadHandler only knows about its own constraint. Before using this pattern, all the constraint logic would have ended up as a big if-else block inside ComputerPlayer.takeAction(), which would have been messy and hard to follow. By moving each constraint into its own class, the code became much easier to read and each class has one clear job. It also made ComputerPlayer a lot cleaner since it just builds the chain and calls handle; it does not need to know anything about how each constraint works internally.

How much did you need to think about the structure of the evolved program once you have figured out which design pattern to use?

Honestly not that much. Once we decided on Chain of Responsibility, the structure basically laid itself out. We knew we needed an abstract handler, concrete handlers for each constraint, and a place to wire the chain together. The only real decision was where to build the chain, and putting it in ComputerPlayer.takeAction() made the most sense since that is where the agent makes its decisions. The pattern gave us a clear blueprint to follow so we spent more time on the actual constraint logic than on figuring out the structure.

How much change was needed in the design and in the implementation?

The changes were not that bad honestly. We added a few new classes which were ConstraintHandler, CardLimitHandler and LongestRoadHandler and the only existing file we

had to touch was ComputerPlayer.java where we built the chain and called it before the R3.2 agent ran. Nothing else needed to change which was nice because we were worried it would break a lot of things but it did not.

6 Design pattern 3

Which design pattern did you choose and why?

The Strategy pattern was chosen for this task to implement the R3.2 requirement. This design pattern falls under the behavioral design pattern category. The R3.2 requirement asks the agent to evaluate a list of rules and pick the one with the highest immediate value, which is exactly what the Strategy pattern is designed for. Each rule represents a distinct behavior with its own evaluation score and execution logic, and they need to be compared and selected at runtime. Encapsulating each rule as a separate class behind a common interface makes this selection clean and extensible.

Explain the design pattern and elaborate on how it fosters cleaner design in this particular software.

Implementing the Strategy design pattern allowed different algorithms to be encapsulated into separate classes. Instead of placing all the logic in one single class, each rule is implemented as its own strategy class. The rules, such as EarnVictoryPoints, BuildSomething, and SpendingCards, implement the Rule interface, and the Agent class stores a list of these rules, evaluates them, and executes the rule with the highest value. Introducing this pattern resulted in a cleaner design, making the code easier to maintain, extend, and modify. ComputerPlayer was minimally modified to add the agent field and delegate takeAction() to the agent, but adding any new rule in the future only requires writing a new class that implements Rule with no further changes needed anywhere else.

How much did you need to think about the structure of the evolved program once you have figured out which design pattern to use?

The structure of the evolved program became much clearer and required less additional design effort once the Strategy pattern was chosen. It defines a clear structure where a common interface represents the strategy, and multiple concrete classes implement different behaviors. In this case, the Rule interface defines the operations, while classes such as EarnVictoryPoints, BuildSomething and SpendingCards implement different rule evaluations. The Agent class simply stores and selects among these strategies by calling evaluate() on each rule and executing the one with the highest value. In case of a tie where all rules return the same score, the Agent picks one at random. Because the pattern provides a predefined organization of responsibilities, most of the program structure naturally follows from the design pattern. The only structural decision that required some thought was whether Agent should live inside ComputerPlayer as a composed field or exist independently, and composition was the right choice since Agent has no meaning outside its player.

How much change was needed in the design and in the implementation?

The structure of the evolved program became much clearer and required less additional

design effort once the Strategy pattern was chosen. It defines a clear structure where a common interface represents the strategy, and multiple concrete classes implement different behaviors. In this case, the Rule interface defines the operations, while classes such as EarnVictoryPoints, BuildSomething and SpendingCards implement different rule evaluations. The Agent class simply stores and selects among these strategies by calling evaluate() on each rule and executing the one with the highest value. In case of a tie where all rules return the same score, the Agent picks one at random. Because the pattern provides a predefined organization of responsibilities, most of the program structure naturally follows from the design pattern. The only structural decision that required some thought was whether Agent should live inside ComputerPlayer as a composed field or exist independently, and composition was the right choice since Agent has no meaning outside its player.

7 Roles and responsibilities

The team members contributed equally to the deliverable.

- Manasvi Bandi: Programming the requirement R3.3 in Task 2.
- Nithya Majeti: Programming the requirement R3.3 in Task 2.
- Lohitashwa Madhan: Doing the requirement R3.2 in Task 3.
- Mohit Gedela: Programming the requirement R3.1 in Task 1.